

NUTRI-SNAP

Nutrition Management Application

Full-Stack Web Development Group Project

React | Node.js | Express.js | MongoDB | REST APIs

Prepared by

Jaturaput	Jongsubcharoen
Gowsith	Thillaiambalam
Aryankumar	Chaudhari
Sukhmanpreet	Singh
Joseph	Abing

Centennial College

2024, December 12

Project Overview

NutriSnap is a group full-stack nutrition management web application developed using React, Node.js, Express.js, REST APIs, and MongoDB. The application allows users to search nutritional information, scan food barcodes, estimate nutritional values using AI, authenticate users, and store personalized nutrition history.

The frontend communicates with the Node.js/Express.js backend through REST API endpoints, while MongoDB provides persistent storage for user-specific nutrition data. The application was deployed using Render with separate frontend and backend services and environment-based configuration.

Client-side GitHub Repository: <https://github.com/Jaturaput-Jongsubcharoen/COMP229-NutriSnap-Client1.git>

Server-side GitHub Repository: <https://github.com/Jaturaput-Jongsubcharoen/COM229-NutriSnap-Server1.git>

Render Client Deployment link: <https://comp229-nutrisnap-client1.onrender.com>

Render Server Deployment link: <https://com229-nutrisnap-server1.onrender.com>

Technology Stack

Frontend: React, JavaScript, HTML, CSS

Backend: Node.js, Express.js, REST APIs

Database: MongoDB, Mongoose

AI & APIs: Google Gemini API, Nutritionix API, Barcode/Quagga integration

Authentication: Token-based user authentication

Deployment & Development: Render, GitHub, Environment Variables

Reflection

Learning through project by Jaturaput Jongsubcharoen

Fist of all, in coding, I learned how to establish the connection between the MongoDB, backend server and frontend server by using a connection string from the MongoDB side, declared as a `MONGO_URI` in order to connect to the backend, and declaring `localhost` from the backend side as a `VITE_BE_URL` to connect to the frontend.

In addition, I learned how to use the http request method such as `GET`, `POST`, and `DELETE`, while working with node.js to store, fetch, and delete the object in the MongoDB. I also used this method to display a username, allowing users to recognize which user ID is currently in use within the application. At this point, I also learned how to modify the code and display only the objects associated with that user ID. Those functions will be used in the History page and some Add-Item page.

Furthermore, I learned how to deploy a full stack app by using a render server. However, I still faced the error while trying to deploy the backend because The issue was hardcoded to access a back end using a direct localhost and also set up a direct localhost in the environment settings on Render

deployment, which was not accessible on the internet. Therefore, I solved it by changing the hardcoded localhost to the environment variable.

Moreover, according to debugging code, while coding the backend and frontend, I faced a lot of errors and the way that I debug my code is by relying on the console tab combined with the Network tab.

Therefore it made me realize the specific problem which occurred from the coding.

Finally, I learned how to communicate with my team members by using GitHub and whatsapp. Using GitHub to push and pull code from team members has helped me work and share smoothly.

Learning through project by Gowsith Thillaiambalam

While Google Gemini API documentation shows the code for NODE.js. The difficulty of the code was to tell my teammates to install the package for front and back-end as many were unaware that it was a package to require downloading. Interestingly, we can switch which model by stating it in the code and we can use it further for testing but we decided to lean on gemini-1.5-flash to make it cost-effective. we use by changing model name which can allow us to use the feature regardless.

In the documentation Gemini documentation states that it has “vision capabilities” which allows it to caption and answer questions about images especially in PNG and JPEG file. However, much of the image API recommendation wasn’t able to transcribe the information, returned the same error which couldn’t make me understand the nature of the errors. Regardless of more time, I think this would have been a solvable issue.

When working on GitHub, the group did amazing especially with the lack of errors in the pull and push records on GitHub but it required careful coordination and assigning each features to a page (to reduce the amount of bugs and errors). The second problem faced was the implementation to history database but thankful I had a team mate that created local history display that was being used for MongoDB collection thus, I built that local history save and used it to implement the feature to save it to MongoDB. to create the basic content to save Despite the time constraint, the application works amazing and can be scaled with more features with the current features working “bug-free”.

Learning through project by Aryankumar Chaudhari

I have had a fun journey working on this React project as it has its own challenges but I have learned a lot of things despite these challenges. How to achieve things with determination and hard work.

This project introduced me to fundamental concepts of react, component based architecture, state management, and use of hooks. I also explored several libraries to enhance the functionality of the code. Moreover, I gained practical experience with github from managing repositories to collaborating with my team.

During my part of the project, I was responsible for adding items into the database. Initially, I faced difficulties in adding data to MongoDB using Mongoose. After overcoming that, I encountered further challenges in fetching the data correctly. Also, I tackled a problem that once a user log in my data was automatically disappearing solved that problem by using console.log to trace the data. Moreover, I

watched a ton of YouTube videos for systematic debugging. These experiences taught me the importance of debugging, testing, and staying patient throughout the process.

An unexpected takeaway was how CSS can significantly impact a project's progress. Improperly structured or conflicting styles can create issues.

Above all, I had the privilege of working with an amazing team. Their support, creativity, and collaborative spirit made the experience not only productive but also enjoyable.

Learning through project – Sukhmanpreet Singh

I have learned so many things so far in this project. I was assigned with the duty to make the login and register page along with its css. Firstly, we decided to complete all the functional requirements for the project including AI, barcode scanning etc. After that, my actual work starts in the 3rd part of the project when I first committed something to the main branch. I faced a lot of challenges throughout the project. That's where my learning starts:

1. Firstly, I don't know how to link the MongoDB with the server as we are using mongoose. So, my Prof. helped me out with this, by letting me know to put the database name in the .env file.
2. Secondly, I learned how to generate the token for the user as when the user joins back it will automatically save the data for him. I even learned how to store data in the mongoDB and then fetching it back during the login operation.
3. Thirdly, I also learned how to make use of GitHub and how to work with it throughout the project such as waiting for others to complete first and then committing everything and seeing the changes.
4. Last but not least, I discovered various debugging skills to make my code functional. In this I also figured out how to accurately design the pages using the css.

Finally, I learned and enjoyed a lot in this project working with different members.

Learning through project - Joseph Abing

Throughout this project I learned so many things. I will highlight the most important ones:

First and foremost, since I am tasked with doing both the Barcode Search page and the Text Search page, I learned the importance of reading and understanding the library and API documentations, specifically the Quagga library and Nutritionix API. My errors came from this part. I learned it is very important to understand the library/API documentation in order to be able to write the correct syntax so that you can connect two different paradigms to be in unison in one single file of code.

Secondly, I learned the intricacies of using a node server to accept connections from the frontend and then connect it to the MongoDB database. Personally, I believe this is one aspect of the project that I need to learn more, the backend as I have stumbled in my code many times on how to save the data fetched from the API to the MongoDB database using the server.js run by node.

Third, and a very important thing that I learned is using GitHub. I was struggling at the beginning, as I was wondering why the code I pushed was not reflected in the deployed site, only to realize that I have to merge my branch to the master branch for it to show on the Render deployed app. Also,

pushing and pulling code from everyone and my work sometimes can run into conflicts. It is important to have patience in debugging conflicts in your code to prevent the app from crashing. Last but not the least, I think communication is very important and is one of the keys to the success of the project. I struggled in part A and B because I worked on the weekends from 6AM to 8:30PM.

Set-up instruction

1. If you check our work from the **Render Client Deployment link**, you can access it right away with any server. **Note:** This application runs on a free server from Render, so please allow a minute as it might take some time to store or fetch data in the MongoDB.
2. If you want to run the application on your own laptop, please clone the repository to your folder and pull the code from both client and server Github repositories.
3. Once you pull it, please open the corresponding folders in the file explorer and then open the terminal.
4. For the server terminal, please type either 'node server.js' or 'npm run dev', and the backend server will run.
5. For the client terminal, please type 'npm run dev', and the frontend server will run.

Using Application Instructions

1. **Main page:** Main page with access to all other pages serving as the main interface of the application.
2. **History page:** In history page. There is a feature to display the and remove the items associated with the specific user, First, the display item function will present all items automatically since the user navigates to the history page. However, the user has to login their user ID in order to see your items that the user has stored recently because there is a condition of checking the token in this function. If there is no item showing up, it means the user hasn't stored any items yet. Therefore, the user has to add some items from the other feature after that you will see the items that you have stored when you come back. Second, there is a delete item function. In order to remove any items from the user's history, the user can click the "remove" button. This button appears next to each item, allowing the user to delete it. Therefore, the user can remove any times flexibly from their history.
3. **Add-Item page:** The Add Item page provides a user-friendly interface for creating and storing customized meal entries into the database. Users can input the meal name, specify custom nutrient details such as carbohydrates, proteins, fats, and calories, and select the meal type (e.g., breakfast, lunch, dinner, or snack) from predefined options. Once the details are entered, the data is securely stored into the database, making it accessible for meal planning.
4. **AI-predict-Image page:** In this page, you will enter your item in any language readable by google gemini ai. Once you enter it, the AI will return an estimate in the prompt. Manually enter the values in

given metrics and select meal types. Once entered, save history. Congratulations! Information now saves history. To check, go to the History Page.

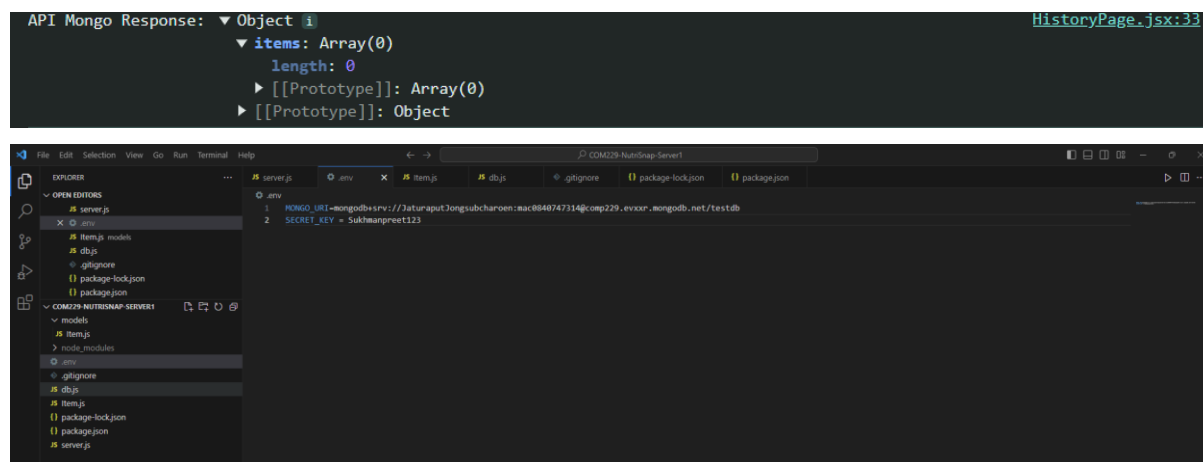
5. Text-Search page: In this page, you will be able to search for any food using the English words and most major languages of the world. Once you click the “Text Search for Food” button on the main page, it will load the Text Search for Food page. In the search bar, the user can enter any name of any food that he wants to know the nutritional value content of. After entering the name of the food to be searched, clicking the “Search” button will automatically search for that food in the food nutrition database of Nutritionix.com. The following values will be displayed to the user: Calories, Protein, Fat, and Carbohydrates. Once a text search is successful, a “Save” button appears to give the user an option to save the searched food nutrients. Once the save button is clicked, the user will see the saved item as a list at the bottom of that page for that session’s most recent searches. The saved food nutrients will also appear on the History Page on the Main Page for that logged user.

6. Barcode page: The Barcode Search feature allows the user to take a picture of any food item that has a UPC/EAN barcode (Canada) in it. This feature is accessible by clicking the “Upload Barcode” button in the homepage. Once clicked, it will load the Barcode scanner page. The user will then click the “Choose File” button in order to select the image file taken of the barcode on any food product that was saved on any device. After selecting the image file, the user clicks on the “Scan Barcode” button. It will then display the detected barcode value to the user. Then it will fetch the specific product and display its nutritional contents to the user. Once a barcode search is successful, a “Save” button appears to give the user an option to save the searched food nutrients. Once the save button is clicked, the user will see the saved item as a list at the bottom of that page for that session’s most recent searches. The saved food nutrients will also appear on the History Page on the Main Page for that logged user.

Problem Found and Solved by Jaturaput Jongsubcharoen

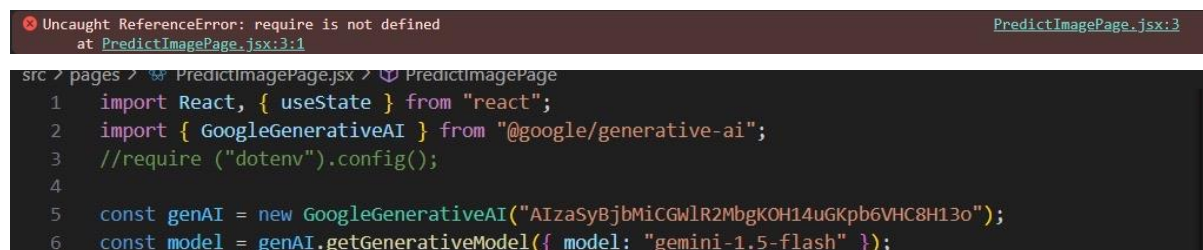
1. Troubleshooting MongoDB Access: Missing Collection in .env File

Regarding the connection to MongoDB, I just know that everyone in my team has to create .env on their server side. Once I shared my MONGO_URI, to let my team members put it into their .env, I forgot to include /testdb, which is my collection name, so they won't be able to access my collection. and I tried to figure out why 3 days ago it was still connected, but 2 days ago, it was not, and it kept showing no error. However, the fetching code worked fine, but no objects retrieving from my MongoDB until I received advice from Prof. Akshay last Friday, I forgot to include /testdb, and I had already pushed it to the server's GitHub repository, so that is why I could not access my MongoDB for a while, but now, it is all good.



2. Fixing Redundant Code Causing Website Display Error

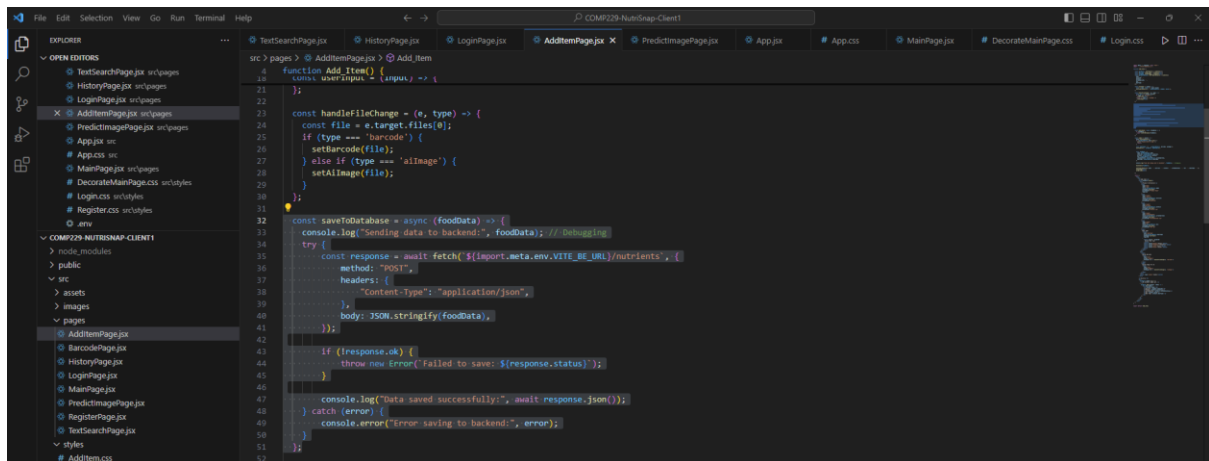
I found one of my members did realize that I have already put `require("dotenv").config();`, and they wrote it repeatedly, so it caused an error of not displaying any features on the website. Therefore, I just put `///
//` as a comment and told my member not to put it twice, so this problem will not occur anymore.



3. Connecting Add-Item Page to Backend and MongoDB

I created the fetching path for the add-item page to connect to the backend server and MongoDB. For the client side, the problem for this page is there is no fetching function to connect to the backend, so I added `fetch(`${import.meta.env.VITE_BE_URL}/nutrients`)`, and created a path `handle-food` function that one of my member has already created. For the server side, I created the `app.post()` to connect

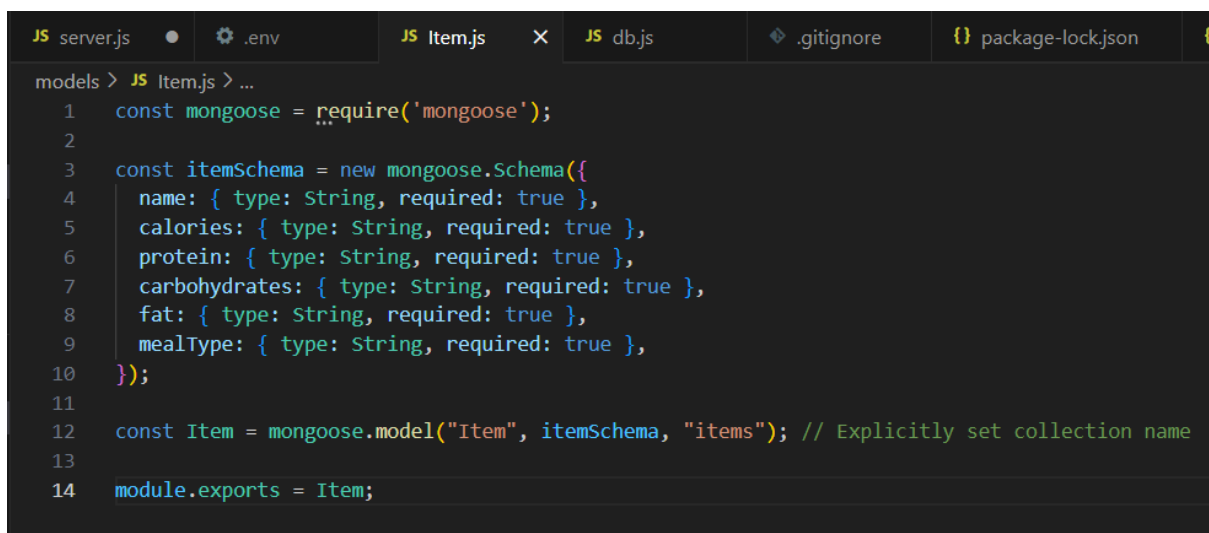
the MongoDB as storing data, and I also created item.js to receive the data from client server and work with the 'app.post()' Indeed, it will turning in it to the variable in the app.post and store it as object in the MongoDB.



```

1  function AddItem(e) {
2    const formData = new FormData(e.target);
3    const file = e.target.files[0];
4    if (file) {
5      const reader = new FileReader();
6      reader.onload = () => {
7        const fileData = {
8          name: file.name,
9          calories: fileData.calories,
10         protein: fileData.protein,
11         carbohydrates: fileData.carbohydrates,
12         fat: fileData.fat,
13         mealType: fileData.mealType,
14         image: reader.result
15       };
16       const foodData = {
17         name: fileData.name,
18         calories: fileData.calories,
19         protein: fileData.protein,
20         carbohydrates: fileData.carbohydrates,
21         fat: fileData.fat,
22         mealType: fileData.mealType,
23         image: fileData.image
24       };
25       const saveToDatabase = async (foodData) => {
26         console.log("Sending data to backend:", foodData); // Debugging
27         try {
28           const response = await fetch(`${import.meta.env.VITE_BACKEND_URL}/nutrients`, {
29             method: "POST",
30             headers: {
31               "Content-Type": "application/json",
32             },
33             body: JSON.stringify(foodData),
34           });
35           if (!response.ok) {
36             throw new Error("Failed to save: " + response.status);
37           }
38           console.log("Data saved successfully:", await response.json());
39         } catch (error) {
40           console.error("Error saving to backend:", error);
41         }
42       };
43       saveToDatabase(foodData);
44     }
45   }
46 }

```



```

1  const mongoose = require('mongoose');
2
3  const itemSchema = new mongoose.Schema({
4    name: { type: String, required: true },
5    calories: { type: String, required: true },
6    protein: { type: String, required: true },
7    carbohydrates: { type: String, required: true },
8    fat: { type: String, required: true },
9    mealType: { type: String, required: true },
10  });
11
12  const Item = mongoose.model("Item", itemSchema, "items"); // Explicitly set collection name
13
14  module.exports = Item;

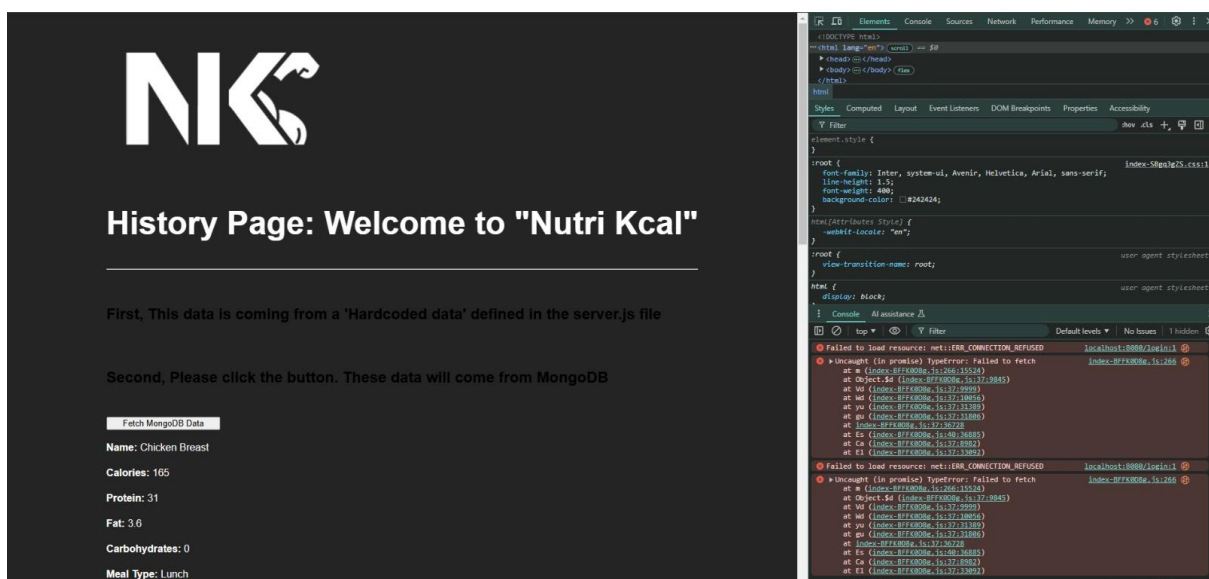
```



```
JS server.js > app.post("/nutrients") callback
58 // Add POST route to save product details to MongoDB
59 app.post("/nutrients", async (req, res) => {
60   try {
61     const { name, calories, protein, carbohydrates, fat, mealType } = req.body;
62
63     // Please don't chage this
64     if (!name || !calories || !protein || !carbohydrates || !fat || !mealType) {
65       return res.status(400).json({ error: "All fields are required" });
66     }
67
68     // Save the data to MongoDB. Please don't chage this
69     const newItem = new Item({
70       name,
71       calories,
72       protein,
73       carbohydrates,
74       fat,
75       mealType,
76     });
77     const savedItem = await newItem.save();
78     res.status(201).json(savedItem);
79   } catch (error) {
80     console.error("Error in /nutrients endpoint:", error);
81     res.status(500).json({ error: "Internal Server Error" });
82   }
83 }
```

4. Fixing Backend Connection Issues and Deployment on Render

I could deploy the application and , for the frontend code, it completely worked fine, but it cannot connect to the backend server. If I did not run my backend terminal, which is server terminal, before I access the Render Client Deployment link. In other words, anything that needs to connect to the backend server will not work until the backend is running on the laptop or desktop you're using to access it.



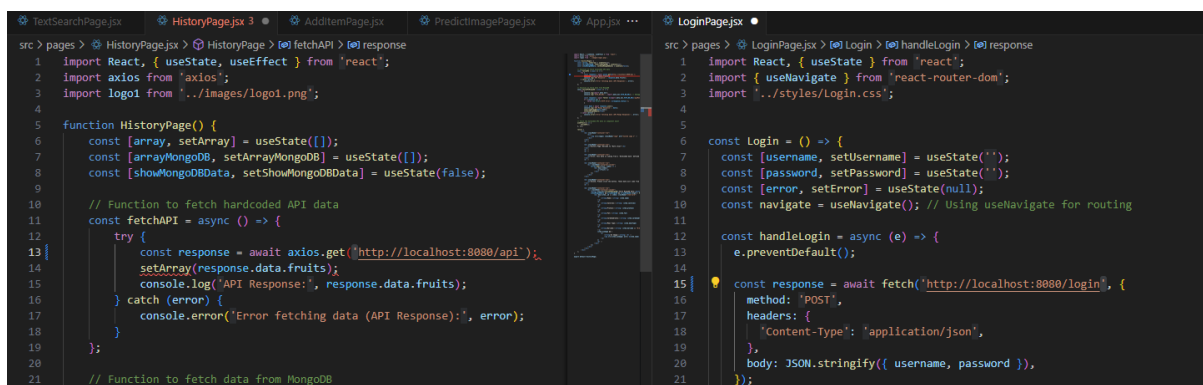
Luckily, I got first advice from Prof. Akshay, that I have to go through the backend setting, and, in the environment setting, I found that I put <http://localhost:8080> into my VITE_BE_URL which made my application cause an error when fetching or storing objects in MongoDB.

```

src > pages > LoginPage.jsx > Login > handleLogin
1  import React, { useState } from 'react';
2  import { useNavigate } from 'react-router-dom';
3  import '../styles/Login.css';
4
5
6  const Login = () => {
7    const [username, setUsername] = useState('');
8    const [password, setPassword] = useState('');
9    const [error, setError] = useState(null);
10   const navigate = useNavigate(); // Using useNavigate for routing
11
12   const handleLogin = async (e) => {
13     e.preventDefault();
14
15     const response = await fetch('http://localhost:8080/login', {
16       method: 'POST',
17       headers: {
18         'Content-Type': 'application/json',
19       },
20       body: JSON.stringify({ username, password }),
21     });
22

```

Once I fixed it a bit, it now works for fetching data from MongoDB. However, the hardcoded data that I defined in the server.js file did not show up, which was weird. It is kind of like connecting directly to MongoDB, bypassing the server. I received a second piece of advice from Prof. Akshay: “[The] login page has the back-end URL hardcoded on line 15. ”, so I need to update all instances of localhost:8080 to use environment variables instead by changing to `import.meta.env.VITE_BE_URL`. that made me realize that the hard code that I put in the server.js was not able to fetch and display on the feature of history page because of using localhost:8080 in the `axios.get()` function, so I just changed it to the environment variables that I have already created. Therefore, I no longer need to run the server side on my laptop before accessing this Render deployment, and anything that needs to connect to the backend server is now displayed successfully.



```

src > pages > HistoryPage.jsx > fetchAPI > response
1  import React, { useState, useEffect } from 'react';
2  import axios from 'axios';
3  import logo1 from '../Images/logo1.png';
4
5  function HistoryPage() {
6    const [array, setArray] = useState([]);
7    const [arrayMongoDB, setArrayMongoDB] = useState([]);
8    const [showMongoDBData, setShowMongoDBData] = useState(false);
9
10   // Function to fetch hardcoded API data
11   const fetchAPI = async () => {
12     try {
13       const response = await axios.get('http://localhost:8080/api');
14       setArray(response.data.fruits);
15       console.log("API Response:", response.data.fruits);
16     } catch (error) {
17       console.error("Error fetching data (API Response):", error);
18     }
19   };
20
21   // Function to fetch data from MongoDB

```

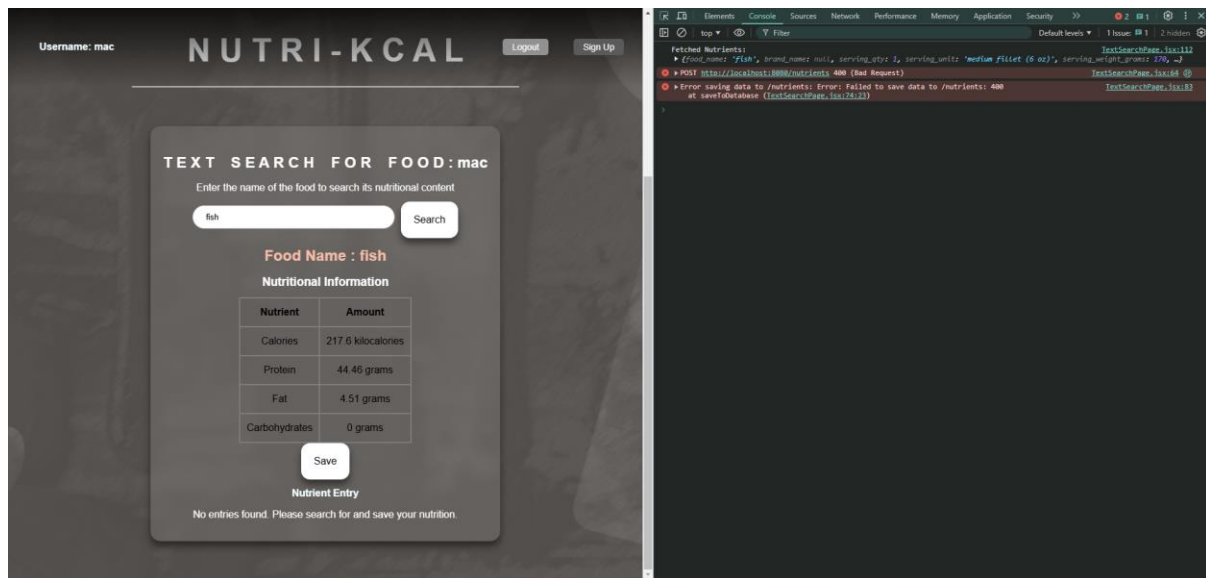
```

src > pages > LoginPage.jsx > Login > handleLogin > response
1  import React, { useState } from 'react';
2  import { useNavigate } from 'react-router-dom';
3  import '../styles/Login.css';
4
5
6  const Login = () => {
7    const [username, setUsername] = useState('');
8    const [password, setPassword] = useState('');
9    const [error, setError] = useState(null);
10   const navigate = useNavigate(); // Using useNavigate for routing
11
12   const handleLogin = async (e) => {
13     e.preventDefault();
14
15     const response = await fetch('http://localhost:8080/login', {
16       method: 'POST',
17       headers: {
18         'Content-Type': 'application/json',
19       },
20       body: JSON.stringify({ username, password }),
21     });

```

5. Debugging and Converting Numeric Values to Strings for Nutrition and Barcode API Data Storage

I modified the code in the text search page and barcode page in order to store the data that is fetching from the nutrition API and scanning from the Barcode API into the MongoDB. I added the `saveToDatabase` function to save nutrient data and also included a save button trigger saving. At first, I got “POST http://localhost:8080/nutrients 400 (Bad Request)”, so I realize that the issue came from the client side, and I found that the main issue of this error is I have to convert numeric values into string by using `.toString()`. because some values in the API are numerical values, so that is how they are sent to MongoDB.



Problem Found and Solved by Aryankumar Chaudhari

1) While developing the React app, I faced challenges when trying to add data to the MongoDB database.

I resolved the issue by carefully defining the MongoDB schema to match the expected data structure and using `async/await` for proper handling of asynchronous operations.

I tested the API using Postman to ensure the data was being sent and stored correctly. Additionally, I updated the frontend code to send properly formatted data and validated inputs to prevent errors. These steps helped me successfully add data to MongoDB.

```
const saveToDatabase = async (foodData) => {
  try {
    const response = await fetch(`${import.meta.env.VITE_BE_URL}/nutrients`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(foodData),
    });

    if (!response.ok) {
      throw new Error(`Failed to save: ${response.status}`);
    }
  } catch (error) {
    console.error('Error saving:', error);
  }
};
```

Problem Found and Solved by Gowsith Thillaiambaam

- 1) Problem with AI page state with changes in save data to history. This resulted in me writing a few console logs and making me discover that data sent to the backend was not properly formatted and returning with error 400. This was quickly changing the format and stringifying fooddata.
- 2) To prevent over use of free API AI Key we can also be replaced with different generative models throughout testing ensuring it remains “free” throughout testing the feature. There is also a limit to each free use api which can be replaced with a different model every time.

Food Nutritional Info

Results:

"ผัดกระเพรา" (Pad Krapow) is a Thai basil stir-fry. There's no single recipe, as ingredients and portion sizes vary widely. Therefore, I cannot give an exact nutritional breakdown. However, a reasonable "estimate" for a typical serving (approximately 1 cup) of Pad Krapow with meat (e.g., chicken or pork) and rice would be: **Calories:** 400-600 kcal **Carbohydrates:** 60-90g (heavily influenced by the amount of rice) **Fat:** 15-30g (depending on the meat and oil used) **Protein:** 20-30g (depending on the meat) This is a broad range. A vegetarian version would have fewer calories and fat, and less protein. A version with a lot of meat and oil would be at the higher end of the range. A smaller portion would naturally have fewer calories, etc.

Manually Edit Values:

Food Name	Calories	Protein (g)	Carbohydrates (g)	Fat (g)	Save to History

Problem Found and Solved by Sukhman (Testing with Gowsith T.)

- 1) Testing revealed the installation of packages is required to run the server. This is not noticeable at first but redesign the error code reveals key words about the package through testing,

```

C:\Windows\system32\cmd.exe
[nodemon] starting 'node server.js'
node:internal/modules/cjs/loader:1228
  throw err;
  ^

Error: Cannot find module 'bcryptjs'
Require stack:
- C:\Users\gowski\OneDrive - Centennial College\Fall 2024\COMP-229 Web Application Development\Group Project Through Mac Github\COM229-NutriSnap-Server1\server.js
    at Module._resolveFilename (node:internal/modules/cjs/loader:1225:15)
    at Module._load (node:internal/modules/cjs/loader:1051:27)
    at Module.require (node:internal/modules/cjs/loader:1311:19)
    at require (node:internal/modules/helpers:179:18)
    at Object.<anonymous> (C:\Users\gowski\OneDrive - Centennial College\Fall 2024\COMP-229 Web Application Development\Group Project Through Mac Github\COM229-NutriSnap-Server1\server.js:6:16)
    at Module._compile (node:internal/modules/cjs/loader:1469:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1548:10)
    at Module.load (node:internal/modules/cjs/loader:1288:32)
    at Module._load (node:internal/modules/cjs/loader:1104:12)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:173:12) {
  code: 'MODULE_NOT_FOUND',
  requireStack: [
    'C:\\Users\\gowski\\OneDrive - Centennial College\\Fall 2024\\COMP-229 Web Application Development\\Group Project Through Mac Github\\COM229-NutriSnap-Server1\\server.js'
  ]
}

Node.js v20.18.1
[nodemon] app crashed - waiting for file changes before starting...

```

```

Microsoft Windows [Version 10.0.22631.4460]
(c) Microsoft Corporation. All rights reserved.

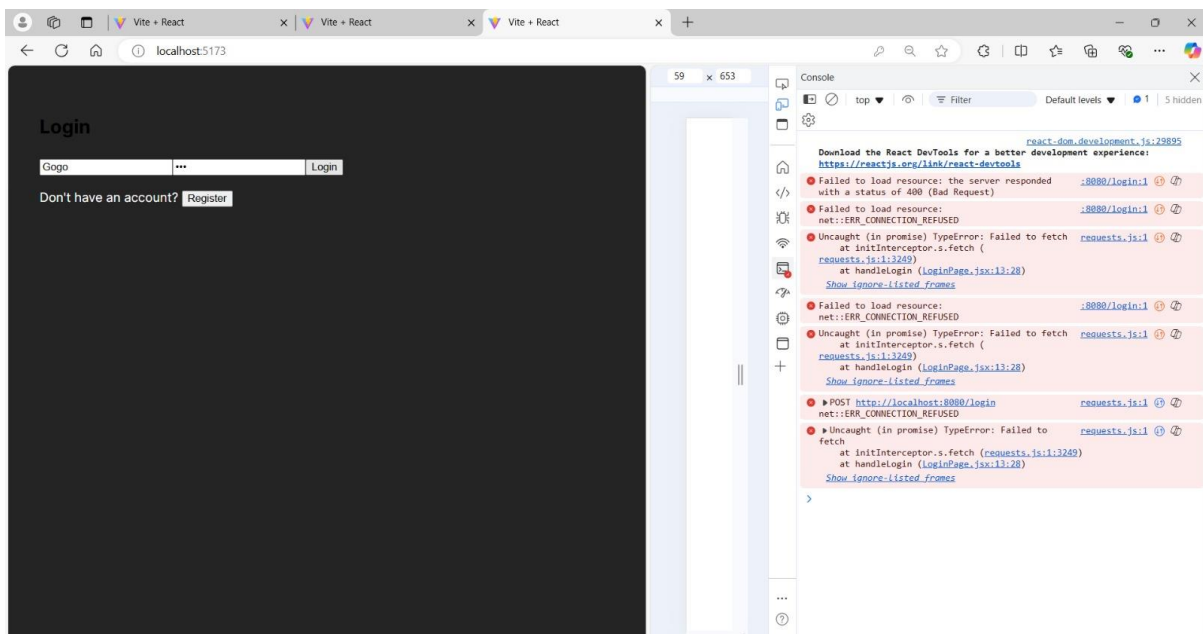
C:\Users\gowski\OneDrive - Centennial College\Fall 2024\COMP-229 Web Application Development\Group Project Through Mac Github\COM229-NutriSnap-Server1>npm run dev

> server@0.1.0 dev
> nodemon server

[nodemon] 3.1.7
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node server.js'
Server running on http://localhost:8080
MongoDB Connected: comp229-shard-00-00.evxxr.mongodb.net

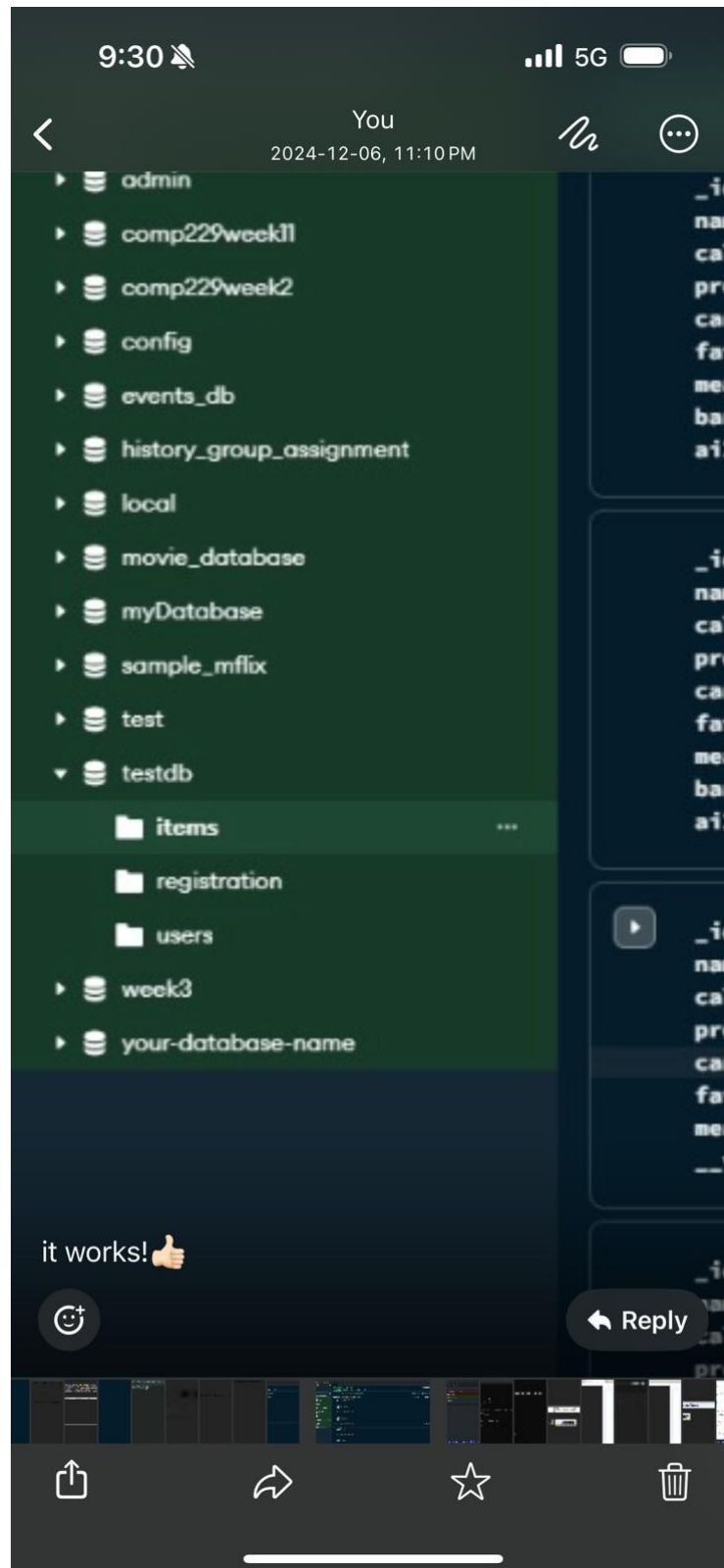
```

- 2) When testing the login in, the secret code isn't being read due to error in the secretkey not being recognized or not being installed. The solution was to test why the secret code won't be recognized due to server env. not being able to find it resulting in reinstallation on tester VSCode and re-uploading to to github.



- 3) Work with Jaturaput to create folders to save information instead of using two databases.

Initially done as a separate branch to ensure the main group was able to work on features without concern of error in login preventing them from working on the application. Retested to ensure viability for deployment.



Problem Found and Solved by Joseph Abing & Jaturaput Jongsubcharoen

1. Error on loading the text search, with inability to access API credentials. After discussing with Jaturaput and after explanation by the professor, the syntax for accessing variables using Vite was changed to

import.meta.env.VITE_NUTRITIONIX_APP_ID
and *import.meta.env.VITE_NUTRITIONIX_API_KEY*

```
const fetchNutritionalInfo = async (barcode) => {  
  try {  
    const response = await axios.get('https://api.nutritionix.com/v2/search/item', {  
      params: {  
        upc: barcode,  
        appId: import.meta.env.VITE_NUTRITIONIX_APP_ID,  
        appKey: import.meta.env.VITE_NUTRITIONIX_API_KEY,  
      },  
    })  
  }  
}
```

2. There was an error in the Client and Server deployment in Render, after checking the Render settings with Jaturaput, we updated the Environment variables for each separate deployment. We placed all environment variables required for each the client and server deployments.
3. Noticed that the Client deployment only was showing the Vite boiler plate template. We asked the Professor what could be the possible problem. We traced the deployment settings with Jaturaput. Jaturaput changed the Build option to “build” still with no effect. But remembering what the Professor discussed, I suggested to Jaturaput to try and change it to “dist”. After clicking the Manual Deploy button and when deployment finished, the Client page was now showing as it should be based on what we pushed to the Github repository.